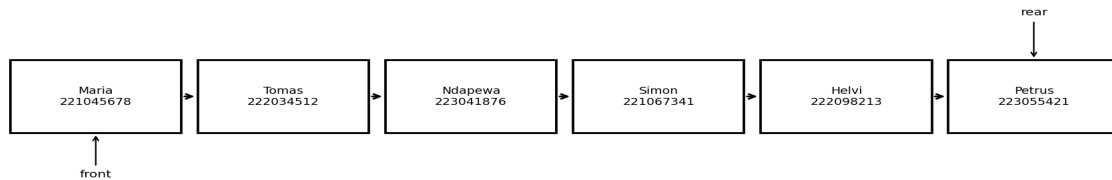


Task A1 — Waiting Line: Queue

Students must be assisted strictly in the order in which they arrive at the service centre. A Queue was implemented from scratch (no `java.util.Queue`) using a linked structure, so the waiting line can grow without a fixed size limit. Required operations: `enqueue()`, `dequeue()`, `peek()`, `isEmpty()`, `displayQueue()`.

1. Queue Diagram

State 1: After 6 student arrivals (enqueue x6)



State 2: After 3 students served (dequeue x3) -> Maria, Tomas, Ndapewa served

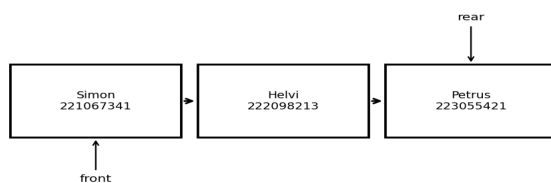


Figure A1.1 — State of the queue after six student arrivals (enqueue), and after the first three students are served (dequeue). New students always join at the rear; service always happens from the front, giving strict First-In-First-Out (FIFO) order.

2. Pseudocode

```
enqueue(student){
    newNode -> data = student
    newNode -> next = NULL
    IF(front == NULL AND rear == NULL) THEN
        front = newNode
        rear = newNode
    ELSE
        rear -> next = newNode
        rear = newNode
    ENDIF
}

dequeue(){
    IF(front == NULL AND rear == NULL) THEN
        DISPLAY "Queue is empty"
    ELSE
        servedStudent = front -> data
        front = front -> next
        IF(front == NULL) THEN
            rear = NULL          // queue became empty
        ENDIF
        RETURN servedStudent
    ENDIF
}

peek(){
    IF(front == NULL AND rear == NULL) THEN
        DISPLAY "Queue is empty"
    ELSE
        RETURN front -> data
    ENDIF
}

isEmpty(){
```

```

    RETURN (front == NULL AND rear == NULL)
}

displayQueue(){
    IF(isEmpty()) THEN
        DISPLAY "Queue is empty"
    ELSE
        temp = front
        WHILE(temp != NULL)
            DISPLAY temp -> data
            temp = temp -> next
        ENDWHILE
    ENDIF
}

```

3. Implementation (key extract)

Implemented in ServiceQueue.java. Internally the queue is a chain of QueueNode objects; 'front' controls dequeue/peek, 'rear' controls enqueue. The full class is included in the source-code submission.

```

public void enqueue(Student student) {
    QueueNode newNode = new QueueNode(student);
    if (isEmpty()) {
        front = newNode;
        rear = newNode;
    } else {
        rear.next = newNode;
        rear = newNode;
    }
}

public Student dequeue() {
    if (isEmpty()) { return null; }
    Student served = front.data;
    front = front.next;
    if (front == null) { rear = null; }
    return served;
}

```

4. Demonstration (program output)

Six students arrive (enqueue x6), then three are served in arrival order (dequeue x3):

```

Enqueued -> 221045678 | Maria   | Registration      | 12 min
Enqueued -> 222034512 | Tomas  | Student Card     | 5 min
Enqueued -> 223041876 | Ndapewa | Fees             | 8 min
Enqueued -> 221067341 | Simon  | Documents        | 4 min
Enqueued -> 222098213 | Helvi  | Academic Enquiry | 10 min
Enqueued -> 223055421 | Petrus | Fees             | 6 min

```

Queue right after 6 arrivals:

```

1. Maria (221045678)   2. Tomas (222034512)   3. Ndapewa (223041876)
4. Simon (221067341)  5. Helvi (222098213)   6. Petrus (223055421)

```

Next to be served (peek): Maria | Registration | 12 min

Serving 3 students (dequeue x3):

```

Served -> Maria      Served -> Tomas      Served -> Ndapewa

```

Queue after 3 students have been served:

```

1. Simon (221067341)   2. Helvi (222098213)   3. Petrus (223055421)

```

5. Why a Queue is Appropriate

The service centre must serve students in the exact order they arrived — the real-world rule is First-Come-First-Served. A Queue enforces this automatically: insertion only happens at the rear (arrival) and removal only happens at the front (being served), so the arrival order can never be disturbed by the data structure itself. A stack would serve the most recent arrival first (LIFO), which would be unfair, and a plain array would need every remaining element shifted left after every dequeue, which becomes expensive as the line grows. Because the queue here is linked rather than a fixed-size array, the waiting line can also grow to any length during a busy period without a 'queue is full' failure.

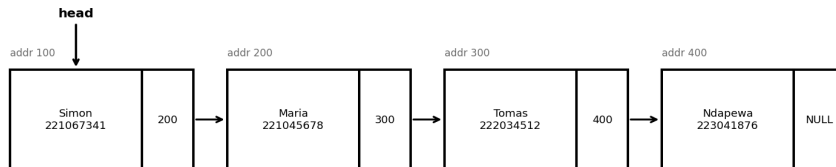
Task A2 — Student Service Records: Singly Linked List

Student service records are stored in a self-built singly linked list. Each node holds one Student record (Student Number, Name, Service Type, Estimated Service Time) plus a 'next' pointer to the following node. Required operations: insertStudent() (at beginning, at end, at a specified position), deleteStudent(), searchStudent(), displayStudents().

1. Linked List Diagrams (before / after)

a) Insertion at a specified position (position 3 — Helvi)

BEFORE: insertAtPosition(Helvi, 3) - 4 nodes in the list



AFTER: new node (addr 500) linked in as the 3rd node - Helvi

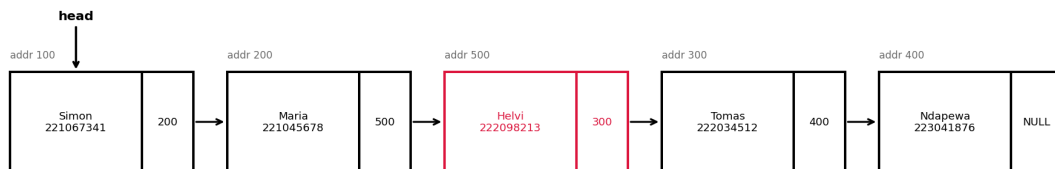
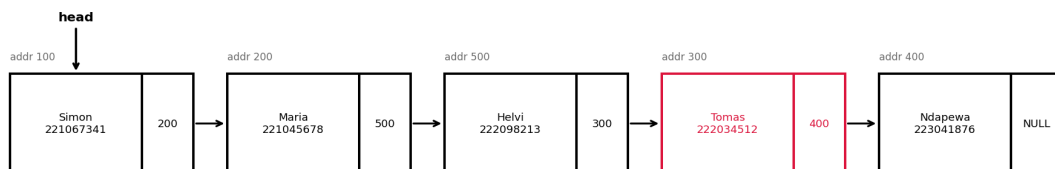


Figure A2.1 — Before: 4 nodes linked Simon → Maria → Tomas → Ndapewa. To insert Helvi as the 3rd node, we walk to the node just before position 3 (Maria), point the new node's 'next' to what Maria was pointing to (Tomas), then point Maria's 'next' to the new node. Only two links change; no data is shifted.

b) Deletion of a node (Tomas)

BEFORE: deleteStudent(222034512) - target node = Tomas (addr 300)



AFTER: Helvi's node (addr 500) now links directly to Ndapewa (addr 400); node 300 freed

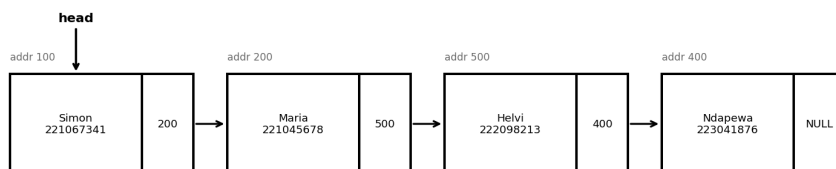


Figure A2.2 — Before: Tomas' node sits between Helvi and Ndapewa. To delete Tomas we move a pointer through the list until we find him, then re-link the previous node (Helvi) directly to the next node (Ndapewa), bypassing Tomas entirely. The node is then freed. After: the list has one fewer node and the chain is unbroken.

2. Pseudocode

```
insertNode(student, mode, position){
    newNode -> data = student
    IF(mode == BEGINNING) THEN
```

```

        newNode -> next = head
        head = newNode
    ELSE IF(mode == END) THEN
        IF(head == NULL) THEN head = newNode
        ELSE
            temp = head
            WHILE(temp -> next != NULL)
                temp = temp -> next
            ENDWHILE
            temp -> next = newNode
        ENDIF
    ELSE IF(mode == POSITION) THEN
        temp = head
        i = 1
        WHILE(i < position - 1 AND temp -> next != NULL)
            temp = temp -> next
            i = i + 1
        ENDWHILE
        newNode -> next = temp -> next
        temp -> next = newNode
    ENDIF
}

deleteNode(studentNo){
    IF(head == NULL) THEN
        DISPLAY "List is empty"
    ELSE IF(head -> data.studentNo == studentNo) THEN
        head = head -> next          // delete the head node
    ELSE
        prevNode = head
        temp = head -> next
        WHILE(temp != NULL)
            IF(temp -> data.studentNo == studentNo) THEN
                prevNode -> next = temp -> next    // bypass the node
                RETURN true
            ENDIF
            prevNode = temp
            temp = temp -> next
        ENDWHILE
        DISPLAY "Student not found"
    ENDIF
}

searchNode(studentNo){
    temp = head
    WHILE(temp != NULL)
        IF(temp -> data.studentNo == studentNo) THEN
            RETURN temp -> data
        ENDIF
        temp = temp -> next
    ENDWHILE
    RETURN NULL    // not found
}

traverseList(){
    temp = head
    WHILE(temp != NULL)
        DISPLAY temp -> data
        temp = temp -> next
    ENDWHILE
}

```

3. Implementation (key extract)

Implemented in StudentLinkedList.java. The full class (with insertAtBeginning, insertAtEnd, insertAtPosition, deleteStudent, searchStudent, displayStudents) is included in the source-code submission.

```

public void insertAtPosition(Student student, int position) {
    if (position <= 1 || head == null) { insertAtBeginning(student); return; }
    Node newNode = new Node(student);
    Node temp = head;
    int i = 1;
    while (i < position - 1 && temp.next != null) { temp = temp.next; i++; }
    newNode.next = temp.next;    // create right link of new node
    temp.next = newNode;        // create left link of new node
}

public boolean deleteStudent(String studentNo) {
    if (head.data.studentNo.equals(studentNo)) { head = head.next; return true; }
    Node prevNode = head, temp = head.next;
    while (temp != null) {
        if (temp.data.studentNo.equals(studentNo)) {
            prevNode.next = temp.next; return true;
        }
        prevNode = temp; temp = temp.next;
    }
    return false;
}

```

4. Demonstration (program output)

Insertion at the end (x3), then at the beginning, then at position 3; followed by a successful search, an unsuccessful search, and a deletion:

List after 3 end-insertions (Maria, Tomas, Ndapewa):

1. Maria(221045678) 2. Tomas(222034512) 3. Ndapewa(223041876)

List after insertion at the BEGINNING (Simon):

1. Simon(221067341) 2. Maria(221045678) 3. Tomas(222034512) 4. Ndapewa(223041876)

List after insertion at POSITION 3 (Helvi):

1. Simon 2. Maria 3. Helvi 4. Tomas 5. Ndapewa

Searching for 223041876 (Ndapewa): Found at position 5 -> Ndapewa | Fees | 8 min

Searching for 999999999 (does not exist): Student 999999999 not found.

Deleting student 222034512 (Tomas): Deleted -> Tomas | Student Card | 5 min

Final list after deletion:

1. Simon(221067341) 2. Maria(221045678) 3. Helvi(222098213) 4. Ndapewa(223041876)

5. Why a Singly Linked List is Appropriate

Student service records are added and removed throughout the day as students register, get served, or have their record corrected, and the total number of records is not known in advance. A singly linked list allocates memory dynamically per node, so the record set can grow or shrink without ever needing to declare a fixed capacity up front, unlike a classical array. Inserting or deleting a record only requires re-pointing one or two 'next' references, which is cheap, rather than shifting every subsequent element as a fixed array would require. Traversal only needs to move forward from head to tail (e.g. to display all records or search for one), so the extra cost of a doubly linked list's back-pointer is not needed here.